



CSE 323/411: Advanced Embedded Systems

Course Content/LO Matrix

LO1	Cognitive Domain	Identifies the concept and tools of RTOS.
LO2	Cognitive Domain	Understands how scheduler and RTOS kernel work.
LO3	Psychomotor Domain	Traces FreeRTOS development process.
LO4	Psychomotor Domain	Develops simple applications on an ARM Cortex M4 architecture using FreeRTOS.
LO5	Psychomotor Domain	Uses documentation for FreeRTOS different functions.
LO6	Affective Domain	Participates in a team to successfully develop a project using FreeRTOS.
LO7	Affective Domain	Asks and listens to different scenarios.

Course Content	LO1	LO2	LO3	LO4	LO5	LO6	LO7
Embedded systems Introduction	X	X					X
The basics or Real Time Operating Systems	X	X					X
FreeRTOS kernel -part 1	X	X	X	X	X		X
FreeRTOS kernel -part 2	X	X	X	X	X		X
FreeRTOS Queues -part 1	X	X	X	X	X		X
FreeRTOS Queues -part 2	X	X	X	X	X		X
Real Time Operating Systems: Task Synchronization, part 1	X	X	X	X	X		X
Real Time Operating Systems: Task Synchronization, part 2	X	X	X	X	X		X
Counting semaphores	X	X	X	X	X		X
Resource management, Mutual Exclusion	X	X	X	X	X		X
Designing a RTOS	X	X	X	X	X	X	X

LOs evaluated in this major task:

Course assessment tool	LO1	LO2	LO3	LO4	LO5	LO6	LO7
Major task (Project)	X	X	X	X	X	X	X

Major course assignment (Project)

Introduction

During this course, you studied FreeRTOS along with its underlying multitasking, synchronization, and resource management techniques. Smart home applications—particularly smart kitchen automation—are becoming increasingly popular by providing safety, energy efficiency, and convenience.

In this project, you are assigned to design a real-time FreeRTOS application to manage a **Smart Kitchen System** that integrates two primary subsystems: a **Smart Lighting System** and a **Smart Oven Control System**. The application will continuously monitor environmental parameters (ambient light and temperature), manage automatic actuation, handle manual user overrides via switches, and report real-time system states to a remote console via UART (displayed on PuTTY).

Objectives

The goal of this project is to design a robust RTOS-based multitasking system with I/O handling, preemptive/round-robin scheduling, and inter-task synchronization. It should have:

- At least four inputs and four outputs (across both subsystems).
- Multiple concurrent, communicating tasks running at various priority levels (including equal priorities to observe Round-Robin time-slicing).
- Effective use of FreeRTOS APIs for task creation, queues, counting semaphores, and mutexes.

Features

In this project, you will design a smart kitchen control system based on a multitasking RTOS architecture. The system will:

- **Smart Lighting Subsystem:**
 - Monitor ambient light levels using a light sensor (LDR/ADC) to automatically turn indoor lights ON or OFF based on a calibrated threshold.
 - Provide a manual lighting override switch that takes precedence over automatic sensor logic.
- **Smart Oven Subsystem:**
 - Monitor oven temperature using a thermal sensor to automatically cut off heating elements when a target temperature is exceeded.
 - Provide a manual oven override switch that can forcefully turn the oven ON or OFF regardless of sensor readings.
- **System Monitoring & Reporting:**
 - Output real-time state updates and system logs to a PuTTY terminal via UART.

Project Outline

1. Inputs and Outputs:

- **Inputs:**
 - Ambient light sensor (ADC/Analog input)
 - Kitchen lighting manual override switch (Digital/Interrupt)
 - Oven temperature sensor (ADC/Analog input)
 - Oven manual override switch (Digital/Interrupt)
- **Outputs:**
 - Kitchen Lighting actuator/LED
 - Oven Heating Element actuator/LED
 - Serial Communication interface (UART Tx to PuTTY)

2. Concurrent Tasks & Priority Design:

- **Task 1: Lighting Control Task (Normal Priority):**
 - Periodically samples the ambient light sensor data.
 - If ambient light falls below the threshold, requests the lighting actuator to turn ON; otherwise, turns it OFF.
 - Sends lighting status updates to the UART Logging Task via a FreeRTOS Queue.
- **Task 2: Oven Control Task (Normal Priority - Equal to Task 1 for Round-Robin execution):**
 - Periodically samples the oven temperature sensor data.
 - If the temperature reaches or exceeds the threshold, safety cut-offs disable the heating element.
 - Sends oven state and temperature alerts to the UART Logging Task via a FreeRTOS Queue.
- **Task 3: User Override Task (Highest Priority):**
 - Monitors the manual override switches for both the lighting and oven systems.
 - Because user interactions require deterministic, low-latency execution, this task runs at the **highest priority**.
 - Overrides automatic sensor logic whenever an active switch event is detected and updates system states immediately.
- **Task 4: UART Logging Task (Lower Priority):**
 - Receives status strings and telemetry data sent from the other tasks via a central message queue.
 - Transmits the system status to the PuTTY terminal over UART.

3. Task Synchronization and Communication:

- **FreeRTOS Queues:** Implement queues to safely pass sensor readings, override events, and formatted state messages from the control tasks to the UART Logging Task.
- **Mutexes:** Implement a FreeRTOS Mutex to guard shared resources (such as the UART hardware peripheral or shared state structure) from concurrent write corruption during context switches.
- **Counting Semaphores:** Use counting semaphores to manage access to bounded system resources or to synchronize event notifications between interrupt handlers (ISR) and high-priority processing tasks.

4. Error Handling:

- Detect and handle hardware fault scenarios (e.g., sensor disconnected/out-of-range readings, switch stuck states).
- Safely force the oven heating element to turn OFF in the event of any thermal sensor fault.
- Output critical error alerts directly to the PuTTY console.

Conclusion

This project will provide hands-on experience with FreeRTOS multitasking, task synchronization, and resource management in a real-time embedded environment. By engineering a dual-subsystem smart kitchen safety and lighting controller, you will gain practical expertise in designing, debugging, and implementing RTOS-based applications on microcontrollers.

Project Evaluation Criteria

- 30% Relevance: Is the project doing the right function (implementation).
- 40% Coherence: How well is the project documentation (design description, final report, code listing, individual reports) to describe the project functionality
- 10% Efficiency: How are the project resources being used. How were the problems solved?
- 10% Effective communication: Are the presentation and discussions effective to describe the project task and implementation.
- 10% Sustainability: Are there lessons learned reported? Are there video presentations to document how the project works?

Project teams

Up to 5 students per team.

Submission deadline (hard deadline)

One week before final exams start

Deliverables

Working on the project should be **in groups up to 5**, which should submit **a compressed file** containing the following deliverables:

- 1.** Source code files (C code & h Files).
- 2.** You should prepare a demo (Video) for evaluation-about 5 minutes long. In the video there should be an explanation of the work of the project. The video should be uploaded to an online server, and the link should be included in the report
- 3.** A technical report in one PDF file containing:
 - a-** It should have title, student names, course code, date of submission, Table of contents, abstract, introduction, body of the report, conclusion/recommendations for future work, and references.
 - b-** The contribution of each member of the group (What did each member do?).
 - c-** Flow charts of the main flow of the program, with detailed word description of all designed code, flow chart, and well documented code listing (using comments)
 - d-** The list of components used and the wiring diagrams.
 - e-** Prototype photos.
 - f-** Problems faced and how you managed to solve them.